Chapter 6

PROBLEM DEFINITION

From practical point of view, linear programming problems have two different representations. The final version of a model created by a modeling system or a special purpose program is presented in some computer readable form, usually in a file. It is called the *external representation.* This file is submitted to a solver which first converts it into an *internal representation.*

The most widely used external representation is the MPS (Mathematical Programming System) format which is considered a de facto industry standard. The internal representation is not standardized. In practice, it is different for each implementation. The only common feature of them is that they all take advantage of the sparsity of the matrix of constraints using some combination of techniques outlined in Chapter 5.

In this chapter we discuss the MPS format and give some hints how it can be processed efficiently.

6.1. The MPS format

The MPS format was introduced by the mathematical programming package of IBM in the 1950s. It reflects the state of computer technology at that time when all primary data had to be entered to a computer on punched cards. These cards had 80 characters each of which could represent a letter, a decimal digit or a symbol (from a limited set). Thus, a natural unit of input was an 80 character long record.

To stay within these limitations bad compromises had to be made. As it will be seen the MPS format is rather awkward and lacks any type of flexibility. Since a very large number of important models have been stored in this format, for compatibility (and convenience) reasons it survived all (otherwise rather vague) efforts to replace it with a more

suitable one. As such it is a must for every serious system to be able to input problems in this format. In practice, solvers usually accept at least two external formats, one of them always being MPS. It is not uncommon that the conversion of the MPS file is done by a separate program which creates an intermediate file and the solver uses it as a direct input.

Because of its importance, the description of the MPS format cannot be missed in a book dealing with the computational aspects of solving LP problems. The basic philosophy of it is correct even today, namely, give only the nonzeros of the problem together with their position to reduce the amount of input data.

As it can be expected from the preliminaries, the MPS format is very rigid. It divides up a record into six strictly positioned fields as shown below (location is the character position within the record):

	Field-1	Field-2	Field-3	Field-4	Field-5	Field-6
Location	2–3	5–12	15–22	25–36	40–47	50–61
Contents	Indicator	Name	Name	Value	Name	Value

The file is vertically structured and consists of sections. Each section is introduced by a section identifier record which contains one of the following character strings starting from position 1:

```
NAME
ROWS
COLUMNS
RHS
RANGES
BOUNDS
ENDATA
```

The sections must appear in this order. `RANGES` and `BOUNDS` are optional.

In the MPS format every entity (constraint, variable, etc.) has a unique identifier (name) consisting of up to 8 characters. The advantages of identifying everything symbolically are obvious. Originally, the MPS format was designed to store a single matrix but several right-hand side, objective, range and bound vectors. They are distinguished by unique names. At solution time the actually used vectors had to be specified. In the absence of the specification the first of each was taken and none of the optional ranges and bounds. The utilization of these possibilities has eroded over the decades and the current practice is to include just one of these vectors which are then automatically selected.

The objective function cannot be given as a separate entity. It is included in the matrix as a non-binding row. The default understanding is that the first non-binding row is considered the objective function. If none is given an error message is triggered.

Each matrix element is identified by a triple in a coordinate system where the coordinates are represented by names like

[column name] [row name] [value]

Similarly, any information relating to a row is identified by the row name. For instance, the structure of defining a b_i element is

[row name] [value]

The same applies to a range value. Obviously, bound entries are identified in a similar structure by column names.

It is to be noted that zero valued entries can also be given, though they most probably will not be stored internally. Such explicit zeros are usually placeholders for modeling purposes.

Comment can be included at any place in any record. A comment is introduced by an $*$ (asterisk) character. Any information, including the asterisk, up to the end of the record is ignored by the input. It is quite typical that comments are included in the MPS file. It is a good practice to give a brief annotation of the problem, date of creation, person responsible, etc. If the problem has already been solved and is meant for other people to test algorithms with the problem then information about the solution can also be included.

The names are expected to start with a non-space character. It is highly recommended to use decimal point in the value fields to avoid an implementation dependent default interpretation of its assumed location when not given.

The individual sections are defined in the following way.

1 `NAME`. This section consists of just this record. The string `NAME` must be in positions 1–4. Additionally, the LP problem can be given a title in positions 15–22. Actual implementations allow the title to be any long up to the end of the record.

  This is the first record of the problem definition. Any information before it is ignored by the input.

2 `ROWS`. In this section the names of all rows are defined. No other name can appear later in the COLUMNS section. Additionally, the type of the constraints are also defined here. The following notations are used.

Row type	Code and meaning
$\leq$	`L` less than or equal constraint
$\geq$	`G` greater than or equal constraint
$=$	`E` equality constraint
Objective	`N` objective row
Free	`N` non-binding constraint

The code can appear in one of positions 2 or 3 (unnecessary freedom!), the row name must be in Field-2.

3 `COLUMNS`. The section identifier must appear in positions 1–7 in a separate record.

This section does two things. First, it defines the names of the columns (variables), second, it defines the coefficients of the matrix including the objective row. As said before, usually the nonzeros are given but some zero placeholders can also appear here. Therefore, the input function must not assume that all given values are nonzero.

The elements of a column can be given in any order (not necessarily in the order of the row names of section ROWS). However, all elements of a column must be given in one group. It is an error to give some elements of a column then some elements of another and again elements of the first.

The data records have the following structure.

Field-2: Column name (maximum 8 characters).
Field-3: Row name (one of those defined in ROWS section).
Field-4: Value of the coefficient.

This structure is repeated for each element of the current column. There is an optional possibility to define two elements in one record. In this case the second [name][value] pair is given in

Field-5: Row name.
Field-6: Value of the coefficient.

The latter can only be used if the first part of the record is also used for defining a matrix element. In this way two elements can be given in a record. The single and double records can be used intermixed. Obviously, a good practice suggests some uniformity.

4 `RHS`. The section identifier must appear in the first three positions of a separate record.

This section gives the values of the right-hand side coefficients. As several right-hand side vectors can be defined here each vector has its own name. The structure of the data records is

Field-2: Name of the RHS vector.
Field-3: Row name.
Field-4: RHS value.

Field-5 and Field-6 can be used the same way as in COLUMNS. The order of giving the RHS coefficients within an RHS vector is insignificant.

Even if all the RHS values are the **RHS** section identifier record must be present.

5 **RANGES**. This is an optional section. If given the string **RANGES** must be in the first six positions of a separate record.

A range constraint has been defined in Chapter 1 as

$$L_i \leq \sum_{j=1}^{\bar{n}} a_j^i x_j \leq U_i,$$

where both L_i and U_i are finite. The range of the constraint is $R_i = U_i - L_i$. One of the values of L_i or U_i is defined in the RHS section which we denote by b_i now. The R_i value of the range is given in the RANGES section. Together with the row type information given in the ROWS section, the L_i and U_i limits are interpreted as follows:

Row type	Sign of R_i	Lower limit L_i	Upper limit U_i
L ($\leq$)	+ or −	$b_i - \|R_i\|$	b_i
G ($\geq$)	+ or −	b_i	$b_i + \|R_i\|$
E (=)	+	b_i	$b_i + \|R_i\|$
E (=)	−	$b_i - \|R_i\|$	b_i

The data records of this section are defined the same way as those of the RHS section. The only difference is the RANGES vector name field.

Field-2: Name of the RANGES vector.
Field-3: Row name.
Field-4: Range value (R_i).

Field-5 and Field-6 can be used the same way as in COLUMNS. The order of giving the range coefficients within a RANGES vector is insignificant.

6 BOUNDS. This is also an optional section. If given the string BOUNDS must be in the first six positions of a separate record.

The definition of the bound vectors is a bit unusual. With a single bound name we can define a pair of lower and upper bounds. It means that a variable can have more than one entry in this section under the same bound name, for example, when both a finite lower and an upper bound are defined.

In the MPS format there is a lot of default values defined which are in force unless they are redefined. First of all, if the BOUNDS section is missing all structural variables are assumed to be of type-2 (0 lower bound and $+\infty$ upper bound).

The data records of this section have the following structure:

Field-1: Bound type.
Field-2: Name of the BOUNDS vector.
Field-3: Column name.
Field-4: Bound value.

Fields 5 and 6 are not used and must be blank (or comment).

Bound type is a two character specifier:

Specifier	Description	Meaning
LO	Lower bound	$\ell_j \leq x_j \ (\leq +\infty)$
UP	Upper bound	$(0 \leq)\ x_j \leq u_j \ (\leq +\infty))$
FX	Fixed variable	$x_j = \ell_j$ (which is $= u_j$)
FR	Free variable	$-\infty \leq x_j \leq +\infty$
MI	Unbounded below	$-\infty \leq x_j \ (\leq 0)$
PL	Unbounded above	$(0 \leq)\ x_j \leq +\infty$

The assumed default bounds are given in parenthesis.

7 ENDATA. This is a stand alone record which terminates the file of the LP problem. It must always be given in the first six positions of a separate record. It is often subject to misspelling. Note it is written with one 'D'.

There are some variations possible with the MPS format which are not particularly relevant to the simplex method. Interested readers are recommended to turn to [Murtagh, 1981] for more details.

Among the deficiencies of the MPS format the most peculiar is the absence of the indicator of the sense of optimization. Therefore, the solver

must somehow be advised whether a given problem is a maximization or minimization.

As an example, we give the following LP problems in MPS format.

$$\begin{array}{rrrrrrl} \max & 4.5x_1 & +2.5x_2 & +4x_3 & +4x_4 & & \\ \text{s.t.} & x_1 & & +x_3 & +1.5x_4 & \leq & 40 \\ 20 \leq & & 1.5x_2 & +0.5x_3 & +0.5x_4 & \leq & 30 \\ & 2.5x_1 & +2x_2 & +3x_3 & +2x_4 & = & 95 \\ & x_1, x_2 \geq 0, & -10 \leq x_3 \leq 20, & 0 \leq x_4 \leq 25. & & & \end{array}$$

It is assumed the names of the rows are `Res-1, Res-2, Balance` and the names of the variables are `Vol--1, Vol--2, Vol--3, Vol--4`. The objective function is called `OBJ`. Other names are defined at the relevant places.

The MPS file is

```
* This a 3 by 4 sample MPS file created 2002.03.15
NAME          Sample Problem
ROWS
 N  OBJ              * Objective function
 L  Res-1
 L  Res-2
 E  Balance
COLUMNS
    Vol--1    OBJ              4.5   Res-1            1.0
    Vol--1    Balance          2.5
    Vol--2    OBJ              2.5   Res-2            1.5
    Vol--2    Balance          2.0
    Vol--3    OBJ              4.0   Res-1            1.0
    Vol--3    Res-2            0.5   Res-3            3.0
    Vol--4    OBJ              4.0   Res-1            1.5
    Vol--4    Res-2            0.5   Res-3            2.0
RHS
    RHS-1     Res-1           40.0   Res-2           30.0
    RHS-1     Balance         95.0
RANGES
    RANGE-1   Balance         10.0
BOUNDS
 LO BOUND-1   Vol--3         -10.0
 UP BOUND-1   Vol--3          20.0
 UP BOUND-1   Vol--4          25.0
ENDATA
```

6.2. Processing the MPS format

Converting an MPS file into internal representation is a major effort. A carefully designed and sophisticated implementation can operate several orders of magnitude faster than a simple straightforward one. There

is little information published about the details of a good design. One exception in the open literature is [Nazareth, 1987] where some important details are given and also pointers to further readings are included. The MSc project report of Khaliq [Khaliq, 1997] with most of the important details is not widely known.

In this section we highlight the complications involved and give guidelines how they can be overcome. The key issue is efficiency and reliability. By the latter we mean that the processing does not break down if errors are encountered in the MPS file but they are detected and appropriate messages are provided that can help the modeler to correct them.

The processing is non-numerical. It mostly consists of creating and using look-up tables. In the internal representation of the LP problem variables and constraints are identified by subscripts. Therefore, the names of the MPS format have to be translated to determine the internal equivalent of, for instance,

[column name] [row name] [value]

or

[row name] [value]

which are a_j^i and b_i.

Processing the MPS format is a surprisingly complicated procedure.

The first thing is to be able to identify the section headers. In the `ROWS` section a table has to be set up that gives the correspondence between row names and row indices. During the setup, duplicate row names have to be identified and ruled out. Also the correctness of the row types has to be checked.

The processing of the `COLUMNS` section is the most complex. When a new column name appears it has to be entered into a table and checked to avoid duplicate column names (elements of a column must be given contiguously). Then the row name is to be looked up in the row names table to get its row index. If it is not on the list the row index cannot be determined thus the element is not included in the matrix. A row name cannot be repeated within a column. It must be checked, too. Finally, the given numerical value also must be tested for syntactical correctness and converted into internal representation (usually using some built-in functions of the programming language, like `read`).

The `RHS` and `RANGES` sections are processed similarly. They involve mostly table look-up, generating error messages if relevant and testing the numerical values. In `RANGES` there is a little extra activity for determining the actual limits according to range table.

The `BOUNDS` section involves more work. First, when a column name occurs it has to be verified by the column names table and its index

retrieved. Then there is some logic needed to sort out what the bound code is (also check for correctness), what it implies, whether there are contradictory entries for a column, etc.

At the end of the processing it is necessary to check whether the required sections were present, objective function was defined. If the answer is 'no' to either of these questions it is a fatal error because the problem is not defined properly. Some non-fatal errors can also be highlighted by warnings, e.g., 'no entry in RHS section' (the section label must be present even in this case).

The efficient operation of these procedures can be ensured by the application of methods of computer science. In this respect the most important basic algorithms are pattern matching, hashing, sorting and binary search. These techniques are discussed in great detail in [Knuth, 1968]. Practical aspects of their use are well presented in several books by Sedgewick, c.f., [Sedgewick, 1990]. The latter book can also be used to implement some critical algorithmic details of the forthcoming versions of the simplex method.